中国科学技术大学



基于 OpenHarmony 操作系统构建离线多模态大模型 APP

参赛队伍: 炸薯丸

队伍成员: 查贝斯　索子骁　王子宁

学　　校: 中国科学技术大学

指导老师: 李永坤　徐伟

完成时间: 2025 年 8 月 16 日

摘要

本项目以“在端侧释放大模型潜能”为核心目标，成功在 OpenHarmonyOS 开发板与实体手机上实现了大语言模型（LLM）和多模态大模型（VLM）的本地化推理。面对计算与内存资源受限的挑战，我们通过对 MindSpore Lite、llama.cpp、MNN 等多种端侧推理框架的深入调研与移植优化，构建出了一套模块化、可扩展、系统级的大模型推理解决方案。

项目采用 MVVM 架构，不仅实现了智能聊天软件 UI、历史对话保存、自定义 Prompt 等应用功能，还支持灵活添加新模型，为用户带来高度可定制化体验。在推理性能方面，团队通过多线程优化与端侧框架对比评测，在有限硬件环境下依然实现了稳定流畅的交互式推理，展现出优秀的工程化能力。

更重要的是，本地化推理方案天然具备隐私安全优势，用户无需依赖云端即可获得多模态大模型的智能体验。结合 HarmonyOS 的“一次开发，多端部署”特性，该框架未来可轻松扩展至 PC、平板、手机等多类终端，构建真正普适的端侧 AI 基础设施。

本项目不仅突破了端侧大模型部署的技术瓶颈，更为本地智能应用生态打开了全新可能。

关键词：OpenHarmony 操作系统、多模态大模型、鸿蒙应用开发

1 项目调研

多模态大模型推理需要较高的计算能力和充足的内存，这两点都不是开发板所具备的，针对比赛要求实现的本地化多模态大模型应用推理，我们在实现开发板运行大语言模型后尝试使用实体手机部署运行多模态大模型应用。

我们希望的研发路线是：通过调研开源的支持端侧部署的框架实现初步的大语言模型推理，在试验成功后逐步实现多模态大模型推理和推理加速。我们选取了较为成熟的端侧推理部署解决方案，下面将分析各个方案并比较方案的优缺点.

1.1 MindSpore Lite

MindSpore 是华为自研的 AI 框架，MindSpore 的轻量化端侧模型，可用于在各种移动端部署机器学习模型。

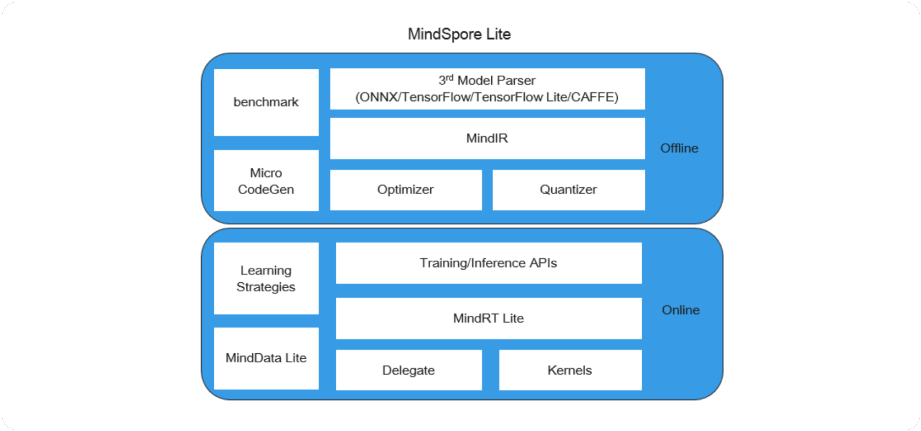


图 1: MindSpore Lite

优点

- 已集成在 openharmony 系统中，可直接通过 APP 进行调用，使用方便
- 支持 CPU、GPU、NPU 异构调度，对 OpenHarmony 优化较好，性能优越

缺点

- Lite 版本并未包含 LLM 模型支持，并未移植 transformer 等部署 LLM 所必须的组件
- Lite 版本无法将 hugging face 的模型转换为所需的模型格式

1.2 Llama.cpp

LLaMA.cpp 项目是开发者 Georgi Gerganov 基于 Meta 释出的 LLaMA 模型（简易 Python 代码示例）编写的纯 C/C++ 版本，用于模型推理。

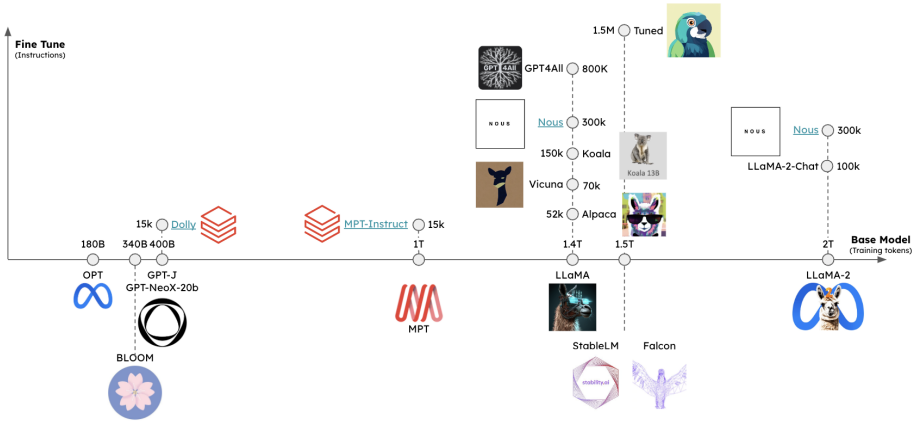


图 2: Llama.cpp 框架

优点

- 专门为部署 LLM 设计，模型支持度广，量化便捷，能方便的支持多种模型
- c++ 编写，可以移植到 OpenHarmony 系统应用中

缺点

- 主要为 PC 端设计，没有针对 openharmony 的优化
- 难以有效利用开发板的 GPU 与 NPU，推理速度慢

1.3 MNN

MNN 是一个轻量级的深度神经网络引擎，支持深度学习的推理与训练。

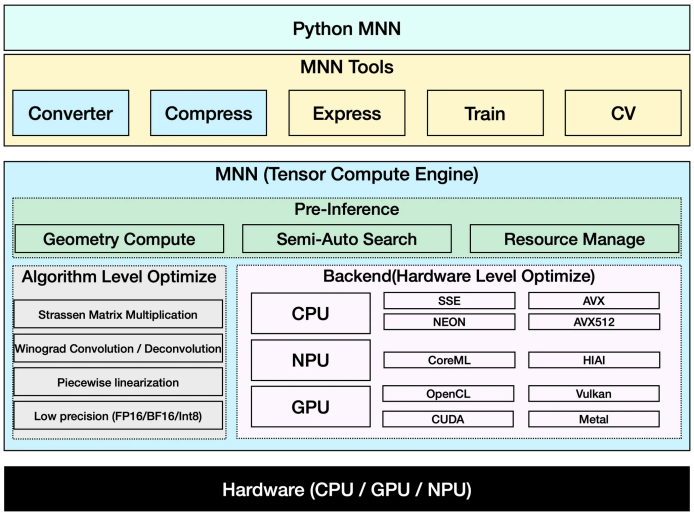


图 3: MNN 框架

在端侧应用 MNN，大致可以分为三个阶段：

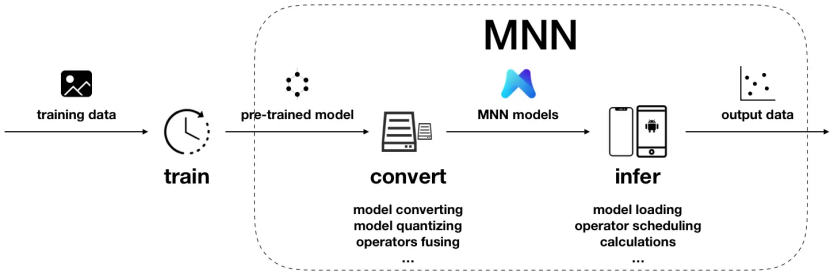


图 4: MNN 推理流程

在项目中我们只需要使用 convert 和 infer 两个阶段的模型处理程序。

优点

- 拥有较为完善的文档和社区支持，本身支持 OpenCV ，可以使用设备 GPU 资源
- 有较为丰富的开源项目

缺点

- 性能优化空间小，目前性能不支持在开发板部署多模态大模型

2 任务分析

项目的基本目标是在装有 'OpenHarmonyOS' 的 'DAYU200' 开发板上开发一款端侧推理的大模型软件，实现了端侧部署推理基础的 LLM 模型，能够稳定理解输入文本并输出文本数据。在内存和硬件资源充沛的设备实现多模态大模型能力。本项目实现的大模型处理框架处于系统层与应用层之间，可以为泛用软件提供端侧推理部署接口，实现端侧推理模块化。

目标一（已完成）：在开发板安装 Open HarmonyOS，能够正常运行测试开发软件

目标二（已完成）：开发一款基于鸿蒙生态的智能聊天软件的应用界面

目标三（已完成）：移植第三方库，在开发板上编译运行大语言模型，实现 LLM 的端侧推理部署

目标四（已完成）：将大语言模型推理与软件整合，实现推理结果输出展示

目标五（已完成）：移植库并优化，实现多模态大模型的端侧推理部署

目标六（已完成）：将模型处理接口对接整合鸿蒙操作系统，实现操作系统级的大模型端侧推理能力

3 设计与实现

根据各级实现目标，我们从大模型处理与部署的设计和软件设计分开并行开发，最终实现了模块化的端侧大模型推理层。再利用比赛提供的开发板，在低内存的状态下实现了最基础的多模态大模型端侧推理部署。

3.1 软件架构

本应用采用 MVVM（Model-View-ViewModel）架构模式来组织应用，通过 model (模型)、view（视图）和 viewmodel（视图模型）之间的分离来实现更好的代码组织和可维护性。

架构图如图 5 所示：

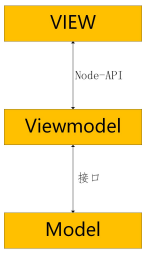


图 5: MVVM 架构图

项目工程结构图 6 所示：

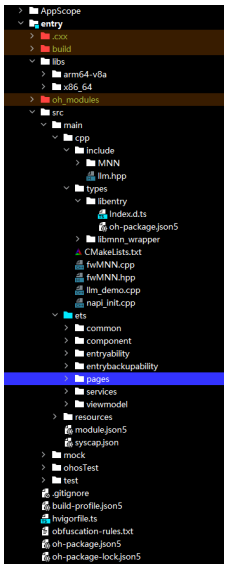


图 6: 工程结构

```
// llama.cpp 的构造与析构函数
llama_cpp::llama_cpp(std::string path){
    model_name = path;
    OH_LOG_INFO(LOG_APP,"load model%{public}s",path.c_str());
    llama_model_params model_params =
        llama_model_default_params();
    model_params.n_gpu_layers = 0;
    //model
```

```
8       model = llama_model_load_from_file(path.c_str(), model_params);
9       if (model == nullptr){
10          OH_LOG_ERROR(LOG_APP,"load model error!");
11          exit(0);
12      }
13      //ctx
14      llama_context_params ctx_params =
            llama_context_default_params();
15      ctx_params.n_ctx = n_ctx_num;
16      ctx_params.n_batch = 128;
17      ctx_params.no_perf = false;
18      ctx_params.n_threads = 4;
19      ctx = llama_init_from_model(model, ctx_params);
20      if (ctx == nullptr) {
21          OH_LOG_ERROR(LOG_APP,"initial context error!");
22          exit(0);
23      }
24      //vocab
25      vocab = llama_model_get_vocab(model);
26      //sample
27      auto sampler_params = llama_sampler_chain_default_params();
28  sampler_params.no_perf = false;
29      sampler = llama_sampler_chain_init(sampler_params);
30      if (sampler == nullptr) {
31          OH_LOG_ERROR(LOG_APP,"initial sampler error!");
32          exit(0);
33      }
34      //llama_sampler_chain_add(sampler,
            llama_sampler_init_greedy());
35      llama_sampler_chain_add(sampler,
            llama_sampler_init_min_p(0.05f, 1));
36      llama_sampler_chain_add(sampler,
            llama_sampler_init_temp(0.8f));
```

```
37      llama_sampler_chain_add(sampler,
            llama_sampler_init_dist(LLAMA_DEFAULT_SEED));
38      OH_LOG_INFO(LOG_APP,"load␣model␣success");
39  }
40
41  llama_cpp::~llama_cpp(){
42      llama_sampler_free(sampler);
43      llama_model_free(model);
44      llama_free(ctx);
45      for(auto &msg:messages){
46          free(const_cast<char *>(msg.content));
47      }
48      delete vocab;
49  }
```

下面，将按照 UI，viewmodel，model 的顺序依次介绍。

3.2 UI 与前端设计

整体采取简约风格，以白色为主、浅色为辅，突出关键元素。

初赛阶段主要借助了第三方库 chatUI 进行了简单定制，效果见下图：

在决赛期间，我们发现第三方库提供的简单接口已经无法满足 app 功能的进一步开发，于是移除了第三方库的依赖，参照其中最核心的消息渲染的思路，抽离关键的 UI 组件，自主实现了更有拓展性、更美观的 UI 界面。

我们的应用包含如下功能与特性：

3.2.1 添加新模型

可以从手机中选择新的模型数据文件并保存在模型列表中。

3.2.2 历史对话

本应用可以保存用户的历史对话，可以记录对话对应的模型，用户的输入和输出。

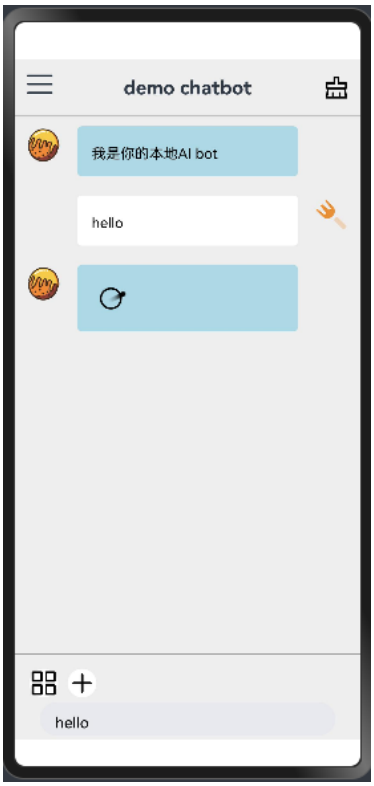


图 7: 初赛阶段的 UI 界面

3.2.3 自定义 prompt

prompt 对于大模型的输出影响极大，支持用户自主修改 prompt。

此外，借助 Harmony 系统的“一次开发、多端部署”理念，后续开发者在本作品的基础上进行简单改进，即可同时在 pc、平板、手机等鸿蒙终端完成部署（UI 层面）。

3.2.4 消息支持 markdown 渲染

借助第三方库，我们实现了对话消息的 markdown 渲染，提升了用户的使用体验。



图 8: 点击设置按钮

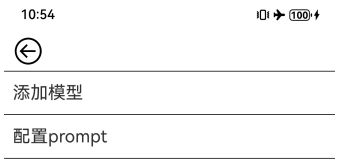


图 9: 设置页面点击添加模型

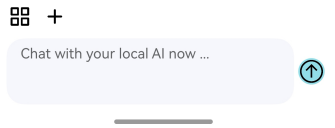




图 10: 拉起模型文件选择页面



图 11: 添加成功

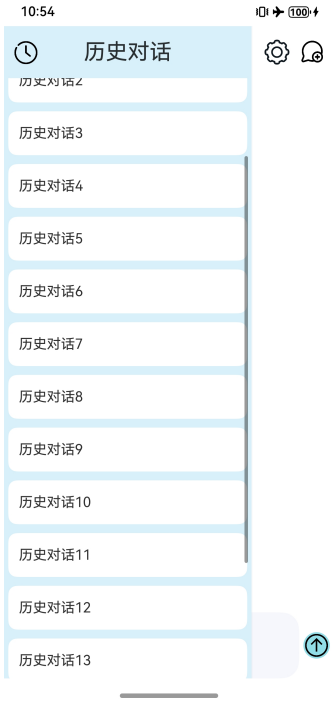


图 12: 侧边栏历史对话列表



图 13: 右滑单个对话框删除或编辑对话名称

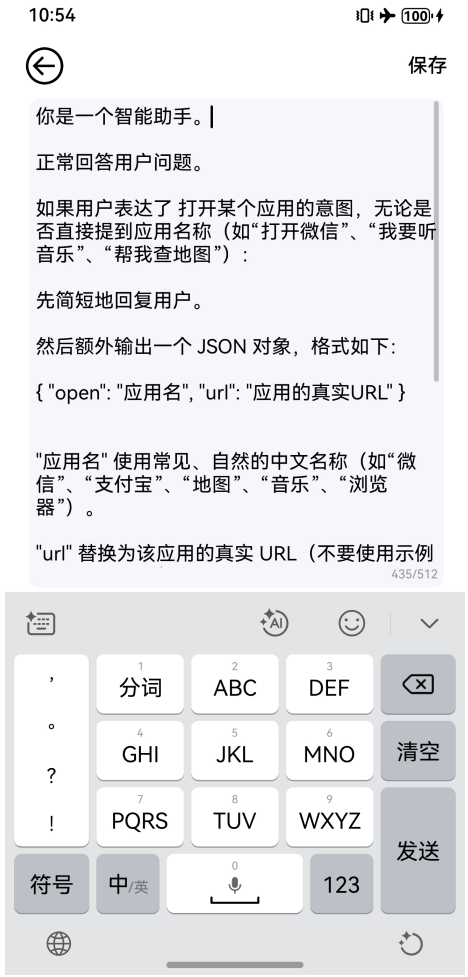


图 14: prompt 示例

10:53

OpenHarmony AI

我是你的本地AI bot

请给我一段带有各种markdown格式的文本

当然可以！以下是一段包含各种 Markdown 格式的文本：

标题层级 1

标题层级 2

标题层级 3

标题层级 4

标题层级 5

标题层级 6

加粗文本

*斜体文本

`代码片段`

~~删除线文本~~

> 这是引用块。
>
> 可以用于引述他人的话或重要提示。

- 列表项 1
- 列表项 2
- 列表项 3
  1. 有序列表项 1
  2. 有序列表项 2
  3. 有序列表项 3

表头1	表头2	表头3
数据1	数据2	数据3
数据4	数据5	数据6

[点击这里访问官网]

```
hello():
    print("Hello, Markdown!")
```

复制

- [x] 已完成
- [ ] 未完成
- [ ] 待处理

图片占位符（实际使用时请替换为真实图片链接）

这是一段完整的 Markdown 示例，包含了大多数常见格式。你可以直接复制粘贴到支持 Markdown 的编辑器中查看效果！

Chat with your local AI now ...

3.2.5 安全添加附件

借助鸿蒙系统的 API，可以安全地从用户本地添加用于多模态大模型推理的附件（图片、音频）。



图 16: 点击按钮添加附件

3.3 viewmodel

viewmodel 层模型推理模块基于 OpenHarmony NDK 开发，采用 C++ 实现。其主要功能包括：

1. **前端交互**：通过 Node-API 与 ArkTS 前端通信
2. **模型对接**：通过标准接口与 model 层交互
3. **数据处理**：处理用户输入，管理推理流程，返回结果

该层实现了双向数据通道，统一了参数传递接口，其与 view 层的交互接口如下：

```
// 加载模型
load_module : (path: string) => void
// 卸载模型
unload_module: () => void
//开始推理
inference_start: (prompt: string,cbFn: (result: string) => void) =>
    void
// 推理结束
inference_stop: () => void
```

为保障线程安全并避免阻塞图形渲染，viewmodel 层采用异步机制与 model 层交互：

1. **线程管理**：使用 pthread 创建独立进程处理加载和推理
2. **资源隔离**：模型操作在后台执行，不影响前端渲染性能

```
// 创建推理线程
void inference_start_thread(napi_env env,napi_ref callback,std::string
    prompt){
    while(waiting)
        std::this_thread::sleep_for(std::chrono::seconds(1));
    waiting = true;
    OH_LOG_INFO(LOG_APP,"start inference");
    napi_value jsCb;
    napi_get_reference_value(env, callback, &jsCb);
    napi_value workName;
```

```
9       napi_create_string_utf8(env, "Inference",
            NAPI_AUTO_LENGTH, &workName);
10      napi_threadsafe_function tsFn;
11      napi_create_threadsafe_function(env, jsCb, nullptr, workName, 0,
            1, nullptr, nullptr, nullptr, callTS, &tsFn);
12  callTs_context *ctx = new callTs_context;
13      OH_LOG_INFO(LOG_APP,"load␣
            model%{public}s",model->test().c_str());
14      model->llama_cpp_inference_start(prompt, [=](std::string
            prompt) -> void{
15              ctx->env = env;
16              ctx->output = prompt;
17  napi_call_threadsafe_function(tsFn, (void*)ctx, napi_tsfn_blocking);
18          });
19      waiting = false;
20  }
```

为实现推理结果的流式返回：

1. **回调封装**：将 view 层回调函数封装为 lambda 表达式
2. **实时传递**：model 层通过回调逐步返回推理结果
3. **界面更新**：前端根据返回数据实时刷新界面

3.4 model

model 层负责加载大模型权重文件，基于前端输入数据进行推理计算，并将最终结果返回前端。由于大模型推理涉及复杂的加载、计算和结果处理流程，需要专用框架支持。同时，推理速度与移动端资源占用直接影响用户体验，因此本队针对 OpenHarmony 平台移植优化了两种主流推理框架：

1. **llama.cpp**：轻量级 C++ 推理框架
2. **MNN**：高性能跨平台推理引擎

并对二者在内存占用、推理延迟等关键指标进行了对比评测。

3.4.1 Llama.cpp 实现方案

llama.cpp 采用 c++ 进行编写。为了将相应的库移植到 openharmony 库中。我们进行了如下工作：

1. 获取 llama.cpp 的源代码
2. 从 openharmony 的官网中获取交叉编译工具链
3. 使用交叉编译工具链对 llama.cpp 进行编译，获得适用于 openharmony 的动态链接库
4. 将编译好的动态链接库放置在 openharmony 的 lib 目录下，并修改 CMakeLists.txt 文件，添加对 llama.cpp 的依赖

为了能够完成推理并将结果返回给 viewmodel 层，我们利用类对 llama.cpp 进行了封装，提供了如下接口：

```
// 负责校验检查模型加载情况
check_model_load
// 辅助处理 llama.cpp 的推理开始
llama_cpp_inference_start
```

其中核心推理启动代码为 **llama_cpp_inference_start** 函数

```
void llama_cpp::llama_cpp_inference_start(std::string prompt,
    std::function<void(std::string)> ref){
    OH_LOG_INFO(LOG_APP,"prompt=%{public}s",prompt.c_str());
    const char * tmpl = llama_model_chat_template(model, /* name
        */ nullptr);
    std::vector<char> formatted(llama_n_ctx(ctx));
    messages.push_back({"user",strdup(prompt.c_str())});
    int new_len = llama_chat_apply_template(tmpl,
        messages.data(),messages.size(),true,formatted.data(),formatted.size());
    if (new_len > (int)formatted.size()){
        formatted.resize(new_len);
        new_len = llama_chat_apply_template(tmpl,
            messages.data(),messages.size(),true,formatted.data(),formatted.size());
    }
```

```
11      if (new_len<0){
12          OH_LOG_ERROR(LOG_APP,"new_len error!");
13          exit(0);
14      }
15      OH_LOG_INFO(LOG_APP,"formatted=%{public}s",formatted.data());
16      std::string new_prompt(formatted.begin() +
            prev_len,formatted.begin()+new_len);
17      //start forward
18      std::string response;
19      const bool is_first = llama_kv_self_used_cells(ctx) == 0;
20      const int n_prompt_tokens = -llama_tokenize(vocab,
            new_prompt.c_str(), new_prompt.size(), NULL, 0, is_first,
            true);
21      std::vector<llama_token> prompt_tokens(n_prompt_tokens);
22      if (llama_tokenize(vocab, new_prompt.c_str(), new_prompt.size(),
            prompt_tokens.data(), prompt_tokens.size(), is_first, true) < 0)
            {
23          OH_LOG_ERROR(LOG_APP,"failed to tokenize the
                prompt");
24          exit(0);
25      }
26      llama_batch batch = llama_batch_get_one(prompt_tokens.data(),
            prompt_tokens.size());
27      llama_token new_token_id;
28      while(true){
29          int n_ctx = llama_n_ctx(ctx);
30          int n_ctx_used = llama_kv_self_used_cells(ctx);
31          if (n_ctx_used + batch.n_tokens > n_ctx) {
32              OH_LOG_INFO(LOG_APP,"context size exceeded");
33              break;
34          }
35          if (llama_decode(ctx, batch)) {
36              OH_LOG_ERROR(LOG_APP,"failed to decode");
```

```
37                exit(0);
38            }
39            new_token_id = llama_sampler_sample(sampler, ctx, -1);
40            if (llama_vocab_is_eog(vocab, new_token_id)) {
41                break;
42            }
43            char buf[256];
44            int n = llama_token_to_piece(vocab, new_token_id, buf,
                  sizeof(buf), 0, true);
45            if (n < 0) {
46                OH_LOG_ERROR(LOG_APP,"failed to convert token
                      to piece");
47                exit(0);
48            }
49            std::string piece(buf, n);
50            OH_LOG_INFO(LOG_APP,"token:%{public}s,n_ctx_used=%{public}d,batch.n_tokens=%{
51            response += piece;
52            ref(response.c_str());
53            batch = llama_batch_get_one(&new_token_id, 1);
54        }
55
56        //change message
57        messages.push_back({"assistant",strdup(response.c_str())});
58    }
```

3.4.2 MNN 实现方案

MNN 本身支持 OpenHarmony，具备对后端 GPU 硬件支持，但同时失去了较多的灵活性。根据 MNN 官方文档，对 MNN 源码进行编译，获得对应.so 依赖库。

将 MNN 对应依赖库和文件导入应用，其具体代码实现与 Llama.cpp 类似。

```
1   void MNN_cpp::MNN_cpp_inference_start(std::string userInput,
        std::function<void(std::string)> ref) {
```

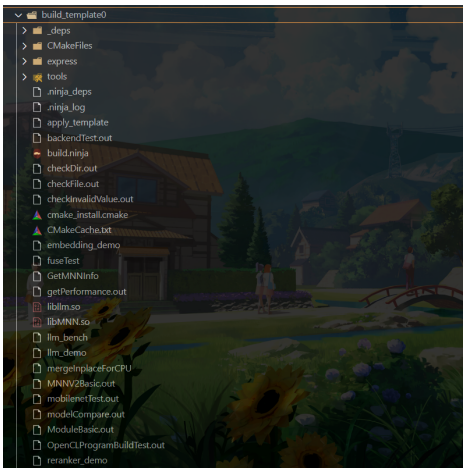


图 17: MNN 构建结果

```
OH_LOG_INFO(LOG_APP,"prompt=%{public}s",userInput.c_str());
std::string response;

MNN_chatMessages.emplace_back("user", userInput);

MNN_llm->set_config(R"({"async":false})");

std::ostringstream nullStream;
MNN_llm->response(MNN_chatMessages, &nullStream, nullptr, 0);
int last_lens = 0;

auto context = MNN_llm->getContext();

while(!MNN_llm -> stoped()) {
    MNN_llm->generate(1);
    response = context->generate_str;
    if (response.length() > last_lens) {
        ref(response);
        last_lens = response.length();
    }
```

```
    }

    MNN_chatMessages.emplace_back("system", response);
    OH_LOG_INFO(LOG_APP, "response=%{public}s",
        response.c_str());
}
```

MNN 在编译视觉和音频能力库后能够处理图片理解和音频识别翻译。

3.5 性能分析

在开发板运行大语言模型时对其进行性能测试，得到数据：

表 1: DAYU200 开发板大语言模型性能

模型名	加载耗时（平均）	推理速率 (token/s)
Qwen3-0.5B	22.0 s	1.12
Qwen1.5-0.5B	24.7 s	0.92

对大语言模型和多模态模型，分别进行性能测试，每轮测试 100 轮对话的平均性能数据，获得如下数据：

表 2: 大模型端侧推理性能测试

model	modelSize	backend	threads	precision	test	t/s
Qwen-VL-Chat-MNN	3.72 GiB	CPU	4	Low	pp512	15.11 ± 3.29
Qwen-VL-Chat-MNN	3.72 GiB	CPU	4	Low	tg128	3.04 ± 0.00

使用 8 线程运行测试：

表 3: 8 线程性能数据

model	modelSize	backend	threads	precision	speed (tok/s)
Qwen-VL-Chat-MNN	3.72 GiB	CPU	8	Low	7.30 ± 2.1515.42 ± 6.30

通过比对数据发现针对目前硬件设备，由于 CPU 核数（4 主核）有限，

更多线程的计算对效率提升不明显，但其运算速度远大于开发板所具备的算力。

4 系统级交互

本队的本地大模型推理方案相比云端方案有显著隐私优势，用户可安全进行深度交互。同时，OpenHarmony 提供丰富的系统接口支持多样化交互。本队在应用中设计了一套接口框架以增强交互能力。

核心接口机制如下：

4.1 Prompt 设计

通过结构化 Prompt 引导模型：

1. 理解用户自然语言请求
2. 识别应用开启意图时输出标准 JSON

设计示例：

'你是一个智能助手。正常回答用户问题。
当用户表达打开应用的意图时（如“打开微信”、“听音乐”），
先简短回复，然后输出JSON：
{ "open": "应用名", "url": "实际URL" }
- 应用名用自然中文（如微信/音乐/地图）
- URL需真实有效

无意图时不输出JSON，确保格式正确'

4.2 数据处理

应用在获得模型输出后：

1. 检测有效 JSON 结构
2. 调用 OpenHarmony 系统 API
3. 执行对应操作（如启动应用）

4.3 具体示例

下图展示了在聊天中透露出意图后，app 拉起了相关应用：

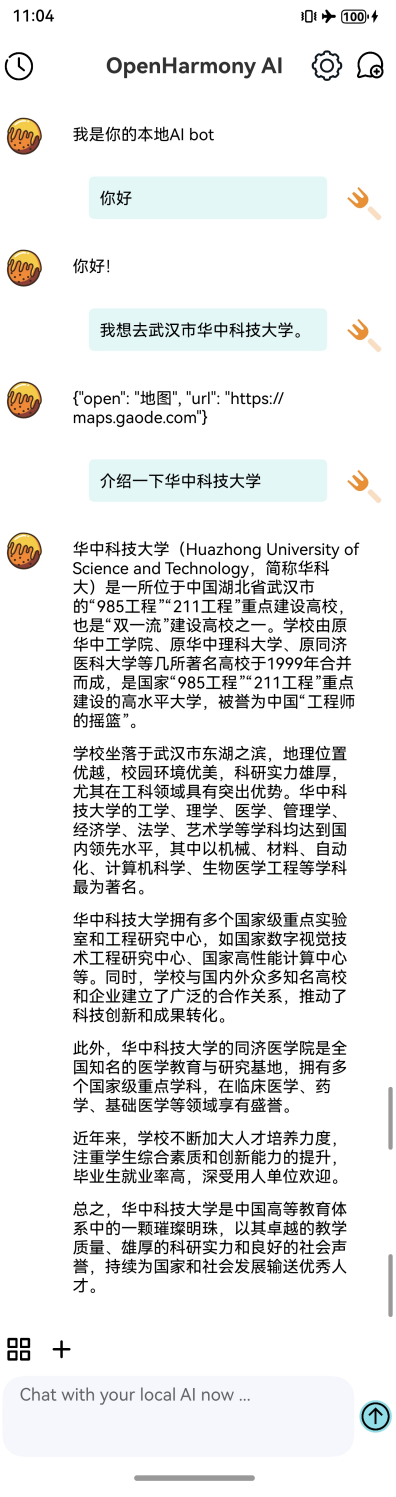


图 18: 透露意图



图 19: 相关应用被拉起

5 产品展望

我们的产品已经完成了大部分本地大模型基础功能，但由于时间有限，还有很多功能等待完善。

5.1 性能优化

目前，llama.cpp 和 MNN 模型的本地模型推理速度可初步满足用户需求，但在用户输入较多时仍需等待较长一段时间。未来，可以考虑将模型 backend 移植到 GPU 或者 NPU 上，以进一步提高推理速度

5.2 系统调用

OpenHarmony 提供了丰富的系统调用，可以提供打开应用，播放音乐，搜索信息等功能。本应用可以与 OpenHarmony 系统进一步结合，实现丰富的功能。

附录

[1]. MNN 框架 https://mnn-docs.readthedocs.io/en/latest
[2]. Llama.CPP 框架 https://github.com/ggml-org/llama.cpp
[3]. markdown 插件 https://gitee.com/luvi/lv-markdown-in